

P0683R1: Default member initializers for bit-fields (revision 1)

Introduction

This paper presents wording for the syntax chosen for default member initializers for bit-fields during the WG21 meeting in Issaquah 2016. It is a follow-on paper to P0187R1 "Proposal/Wording for Bit-field Default Member Initializer Syntax" by Andrew Tomazos.

In a nutshell, the proposal with the most consensus was to support

```
struct S {  
    int x : 8 = 42;  
};
```

with the proviso that parsing ambiguities such as

```
int a;  
struct S2 {  
    int y : true ? 8 : a = 42;  
    int z : 1 || new int { 0 };  
};
```

are resolved with a max-munch rule.

Discussion

The grammar for the *member-declarator* of a bit-field (12.2.4 [class.bit]) is

identifier_{opt} attribute-specifier-seq_{opt} : constant-expression

In turn, a *constant-expression* is a *conditional-expression* (see 8.20 [expr.const]), therefore a = cannot possibly be part of that *constant-expression* at the top level. This makes the motivating use-case work with a max-munch rule, because parsing of the *constant-expression* will stop when the = is encountered.

The ambiguous cases shown above are both parsed as convoluted *constant-expressions*, so neither S2::y nor S2::z have a default member initializer, preserving backward compatibility. The meaning can be changed by applying parentheses:

```
int a;  
struct S2 {  
    int y : (true ? 8 : a) = 42;  
    int z : (1 || new int) { 0 };  
};
```

Wording changes

In 12.2 [class.mem] paragraph 1, change

member-declarator:
declarator virt-specifier-seq_{opt} pure-specifier_{opt}

declarator brace-or-equal-initializer_{opt}
identifier_{opt} attribute-specifier-seq_{opt} : constant-expression brace-or-equal-initializer_{opt}

Add after 12.2 [class.mem] paragraph 7:

In a *member-declarator*, an = immediately following the *declarator* is interpreted as introducing a *pure-specifier* if the *declarator-id* has function type, otherwise it is interpreted as introducing a *brace-or-equal-initializer*. [Example: ...]

In a *member-declarator* for a bit-field, the *constant-expression* is parsed as the longest sequence of tokens that could syntactically form a *constant-expression*. [Example:

```
int a;  
const int b = 0;  
struct S {  
    int x1 : 8 = 42;           // OK; "= 42" is brace-or-equal-initializer  
    int x2 : 8 { 42 };         // OK; "{ 42 }" is brace-or-equal-initializer  
    int y1 : true ? 8 : a = 42; // OK; brace-or-equal-initializer is absent  
    int y2 : true ? 8 : b = 42; // error: cannot assign to const int  
    int y3 : (true ? 8 : b) = 42; // OK; "= 42" is brace-or-equal-initializer  
    int z : 1 || new int { 0 }; // OK; brace-or-equal-initializer is absent  
};
```

-- end example]

In 12.2.4 [class.bit] paragraph 1, change

A *member-declarator* of the form

identifier_{opt} attribute-specifier-seq_{opt} : constant-expression brace-or-equal-initializer_{opt}

specifies a bit-field; its length is set off from the bit-field name by a colon.